DOCUMENTAÇÃO DE IMPLEMENTAÇÃO / VERSÃO 0.1

MicroLearn

Sprint 1 — Fundação técnica, autenticação e primeira interface

Relatório de execução técnica

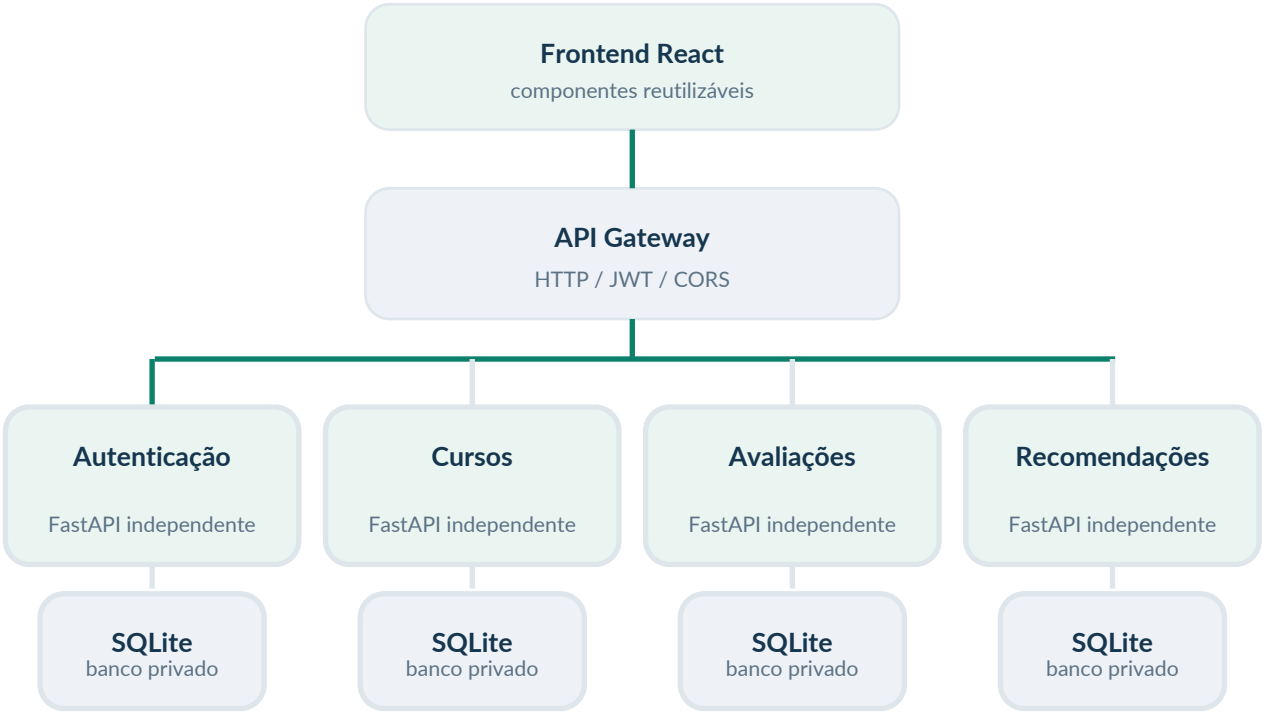
Este documento registra a implementação inicial do Ambiente Virtual de Aprendizagem MicroLearn. O incremento estabelece a infraestrutura de microsserviços, implementa a autenticação e prepara uma interface web baseada em componentes React.

Resultado da Sprint 1	Situação verificada
API Gateway com encaminhamento HTTP e JWT	Implementado e testado
Cadastro, login, perfil e administração no Auth Service	Implementado e testado
Serviços de Cursos, Avaliações e Recomendações	Processos e armazenamento criados; negócios pendentes
Frontend React para início, cadastro, login e painel	Código escrito; build não verificado neste ambiente
Docker Compose e imagens separadas	Configuração criada; execução Docker ainda não validada
Suíte de testes Python e smoke HTTP	10 testes aprovados + smoke aprovado

Escopo deliberado: nesta sprint os serviços de Cursos, Avaliações e Recomendações respondem HTTP 501 para funções de negócio. O funcionamento dessas funcionalidades será implementado nas Sprints 2 e 3.

1. Arquitetura executável

A aplicação separa a interface, o ponto de entrada HTTP e quatro microsserviços de negócio. Cada serviço tem código próprio e banco SQLite exclusivo. A figura representa a arquitetura prevista no Docker Compose.



O gateway recebe requisições do navegador em /api/v1, valida o JWT em rotas protegidas e encaminha os pedidos para serviços internos. Cada domínio executa em processo isolado e é empacotado em contêiner próprio.

Tecnologia	Função
Python / FastAPI	APIs REST e documentação OpenAPI
SQLite	Persistência do Auth e bases separadas dos demais domínios
React / Vite	Componentes e navegação web
Docker Compose	Construção e orquestração dos seis contêineres
httpx / PyJWT	Chamadas de rede e tokens assinados

2. Funcionalidades implementadas

2.1. Autenticação e administração

Endpoint	Resultado / controle
POST /api/v1/auth/register	Cadastro público apenas de aluno. E-mail único; senha com hash PBKDF2.
POST /api/v1/auth/login	Validação de credenciais e geração de JWT com expiração.
GET /api/v1/auth/me	Perfil do usuário ativo, obtido no banco de Autenticação.
GET /api/v1/auth/users	Listagem restrita a administrador.
PATCH /api/v1/auth/users/{id}	Alteração de papel ou status somente por administrador.
GET /api/v1/status	Consulta agregada de saúde de todos os microsserviços.

2.2. Componentes reutilizáveis da interface

Navbar e AppShell fornecem navegação e estrutura comum; Alert apresenta mensagens; ProtectedRoute protege o painel; AuthContext mantém o estado da sessão; apiClient centraliza as requisições HTTP.

2.3. Proteções implementadas

Senhas protegidas com PBKDF2-HMAC-SHA256, sal aleatório e 600 mil iterações; JWT HS256 com emissor, público, criação e expiração; restrições administrativas verificadas no Auth Service; ausência de banco compartilhado; CORS de origem configurável.

Os tokens em sessionStorage são apropriados apenas ao protótipo didático. O projeto ainda não possui revogação de JWT, redefinição de senha, HTTPS ou limitação de tentativas de login.

3. Testes e evidências

A validação abaixo foi executada no ambiente de preparação do projeto; não representa uma execução do Docker no computador do aluno.

Verificação executada	Resultado observado
python -m pytest -q	10 testes aprovados
Smoke test com 5 processos Python	Gateway + 4 serviços online
Cadastro pelo HTTP da Gateway	201 Created
Login pelo HTTP da Gateway	200 OK; JWT retornado
Perfil com JWT	200 OK; perfil student
Perfil sem JWT	401 Unauthorized
GET /api/v1/courses	501 Not Implemented, como previsto
Arquivos JavaScript e JSX	12 arquivos sintaticamente válidos pelo parser TypeScript
docker-compose.yml	YAML interpretado sem erro

3.1. Validações pendentes

O Docker não estava disponível neste ambiente, e a instalação npm não terminou. Por isso, não foi possível certificar o build do frontend, o funcionamento dos seis contêineres ou as telas no navegador. O roteiro do README permite executar essas verificações no Windows.

O relatório registra evidências presentes em docs/pytest_results.txt e docs/smoke_test_results.txt. As capturas do Docker Compose devem ser acrescentadas depois da execução no computador do desenvolvedor.

4. Execução no Windows e próximos passos

A execução foi preparada para Docker Desktop com PowerShell. O projeto contém um arquivo .env.example, que deve ser copiado para .env e preenchido com uma chave JWT forte.

Etapa	Comando ou operação
1	Copy-Item .env.example .env
2	python -c "import secrets; print(secrets.token_urlsafe(48))"
3	notepad .env (inserir o segredo JWT gerado)
4	docker compose up --build -d
5	docker compose ps
6	Abrir http://localhost:5173
7	Abrir http://localhost:8000/api/v1/status

Se desejado, configure ADMIN_EMAIL e ADMIN_PASSWORD no .env para criar uma conta administrativa. O administrador pode converter um aluno em professor usando PATCH /api/v1/auth/users/{id}.

5. Planejamento de continuidade

Próxima entrega	Implementação prevista
Sprint 2	CRUD de cursos, aulas, publicação e matrícula; telas de catálogo e professor.
Sprint 3	Questionários objetivos, correção, notas e recomendações por desempenho.
Sprint 4	Integração ponta a ponta, revisão de segurança, testes, relatório e GitHub.

Referências de trabalho: proposta do MicroLearn, requisitos (Etapa 2), arquitetura (Etapa 3), Product/Sprint Backlogs (Etapa 4), roteiro da disciplina e código-fonte entregue no ZIP.